

Project 2 Function Generator

Sean Tracy, Krishna Sadhanala
CPE 316
Fall Quarter
11/17/2025

Behaviour Description

This project used the keypad to allow user input to generate a set range of frequencies and waveforms via MCP4921 DAC output. The 1-5 keys on the keypad change the frequency of the output corresponding to 100-500Hz, the 6-9 keys change the type of waveform between sine, triangle, sawtooth, and square respectively, and the *,0,# keys change the duty cycle (only in use for the square waveform).

Schematic

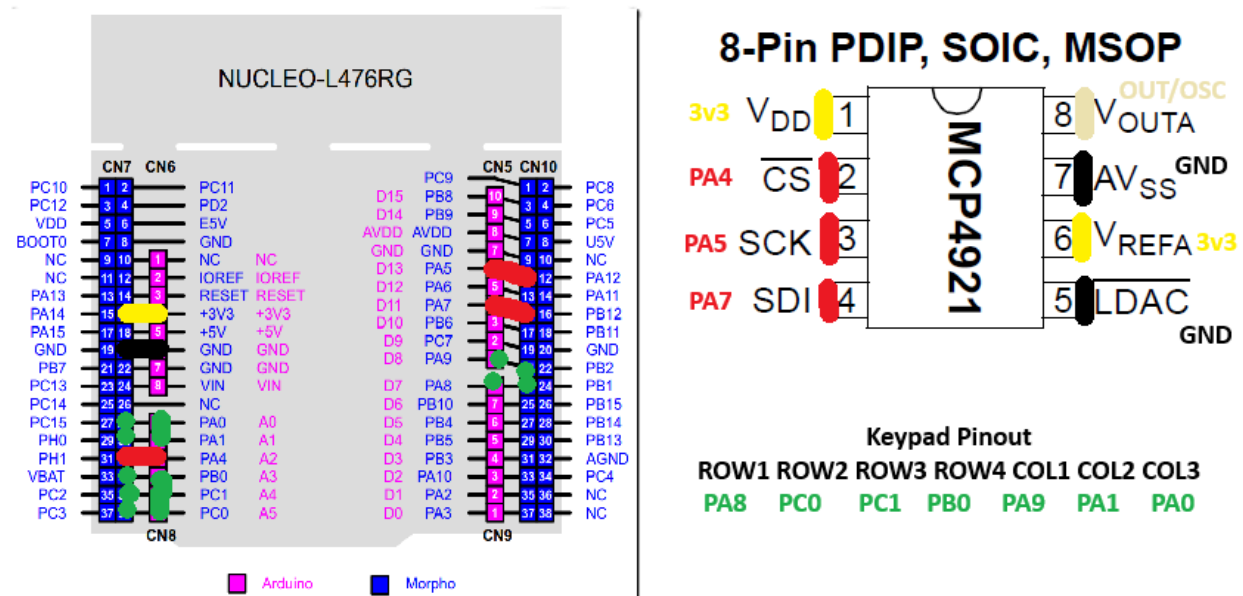


Figure 1: Nucleo board pinouts to peripherals

Software Description

As stated above, this project requires 12 different options and many different “states” where the frequency, waveform, or duty cycle differ. In order to simplify our main while loop, we kept most of the code outside of this loop and kept it to 2 lines. One that checks which key is pressed, and another to call the function that changes the desired setting based on the pressed button. The function that we created for this takes the keypad input and changes a global variable that holds and indicates which waveform, frequency, and duty cycle we want. By only changing the 3 global variables, we simplified the code down to only a few switch statements that check for wave type, frequency, and duty cycle. For the wave generation, we created lookup tables with a size of 240 for each wave form; using math functions in initialization to create the arrays. Each point is generated by using the interrupt handler to increment through the look-up table whenever the TIM2 counter reaches the desired ARR value. For square

waves, we used the CCR1 register to set the square wave low at the desired interval based on the set duty cycle.

Questions

1. The most obvious way to increase responsiveness would be to make it an interrupt instead of polling to see which key was pressed. Another way that could add onto this would be to create a button debouncer instead of using a HAL_Delay to account for button bounce.
2. The limiting factor of how many points per period we can output is the time it takes for a DAC_write to occur, as if we try to output too many points per period DAC_writes could end up lost and the output would have missing datapoints. Technically if DAC_write was instantaneous, then the limiting factor would be the MCU clock/interrupt handle time.
3. It is a technique for generating precise waveforms digitally by using a numerically controlled oscillator (NCO) that increments a phase accumulator each clock cycle. In our function generator, we increment our wave value whenever the timer updates/hits the ARR value, but direct digital synthesis changes the phase value in the accumulator. Additionally, DDS usually has a low pass filter to smooth out the final output waveform.
4. Sean: The most challenging part of this project was figuring out small timing nuances that were hard to diagnose. We ran into a lot of small, weird issues that ended up being solved by things like changing the clock speed or stopping the clock while setting the ARR to a new value. The most important thing I learned was that sometimes, the issue is not with your logic, but instead in the logistics. I learned that I also should test and debug higher level stuff like the clock speed.

Krish: Understanding how code being ran without dependence on timers or interrupts operates at a variable, unpredictable pace compared to code that is tied to timers/interrupts and how not unifying that can cause a multitude of issues hard to diagnose was the most challenging part of this project.

Appendices

main.c

```

/* USER CODE BEGIN Header */
#include "main.h"
#include "DAC.h"
#include "Keypad.h"
#include <math.h>
#include <stdio.h>
#define sine 6
#define triangle 7
#define sawtooth 8
#define square 9
#define TABLE_SIZE 240
//Timer at 32MHz
#define Hz100 (32000000 / (100 * TABLE_SIZE)) - 1 //create ARR Value for non-square waves
#define Hz200 (32000000 / (200 * TABLE_SIZE)) - 1
#define Hz300 (32000000 / (300 * TABLE_SIZE)) - 1
#define Hz400 (32000000 / (400 * TABLE_SIZE)) - 1
#define Hz500 (32000000 / (500 * TABLE_SIZE)) - 1
//ARR Values for square waves since no table is used
#define SqHz100 (32000000 / 100) - 1
#define SqHz200 (32000000 / 200) - 1
#define SqHz300 (32000000 / 300) - 1
#define SqHz400 (32000000 / 400) - 1
#define SqHz500 (32000000 / 500) - 1
uint16_t sine_table[TABLE_SIZE]; //Initialize look up tables
uint16_t sawtooth_table[TABLE_SIZE];
uint16_t triangle_table[TABLE_SIZE];
volatile int wave_index = -1; //global index variable
volatile int hold_duty_cycle = 50; //Duty cycle global variable
volatile int hold_waveform = square; //Used to track waveform
volatile int hold_freq = 1; //Used to track and hold frequency

void set_ARR(int ARR_value){ //Stop timer to set ARR, avoid timing issues
    TIM2->CR1 &= ~TIM_CR1_CEN;
    TIM2->ARR = ARR_value;
    TIM2->CNT = 0;
    TIM2->EGR = TIM_EGR_UG;
    TIM2->CR1 |= TIM_CR1_CEN;
}

void set_CCR(void){ //Stop timer to set CCR, avoid timing issues
    TIM2->CR1 &= ~TIM_CR1_CEN;
    TIM2->CCR1 = (uint32_t)((((TIM2->ARR + 1) * hold_duty_cycle) / 100) - 1);
    TIM2->CNT = 0;
    TIM2->EGR = TIM_EGR_UG;
    TIM2->CR1 |= TIM_CR1_CEN;
}

```

```

void set_freq(int freq){ //Set frequency depending on hold freq and hold waveform
    int ARR = 0;
    if (hold_waveform == square){ //Check to pull square wave or non-square wave
frequencies
        switch (freq){
            case 1: ARR = SqHz100;
                break;
            case 2: ARR = SqHz200;
                break;
            case 3: ARR = SqHz300;
                break;
            case 4: ARR = SqHz400;
                break;
            case 5: ARR = SqHz500;
                break;
        }
    } else {
        switch(freq){
            case 1: ARR = Hz100;
                break;
            case 2: ARR = Hz200;
                break;
            case 3: ARR = Hz300;
                break;
            case 4: ARR = Hz400;
                break;
            case 5: ARR = Hz500;
                break;
        }
    }
    set_ARR(ARR); //Set ARR to designated value
}

void square_wave(){ //Hold wave form until key is pressed
    wave_index = -1;
    HAL_Delay(200); //Ensure that duty cycle is not accidentally increased by 2
    while(check_press(0) == -1);
}

void sine_wave(){ //Hold wave form until key is pressed
    wave_index = 0;
    TIM2->CCR1 = 0;
    while(check_press(0) == -1);
}

void sawtooth_wave(){ //Hold wave form until key is pressed
    wave_index = 0;
    TIM2->CCR1 = 0;
    while(check_press(0) == -1){
    }
}

void triangle_wave(){ //Hold wave form until key is pressed

```

```

    wave_index = 0;
    TIM2->CCR1 = 0;
    while(check_press(0) == -1){
    }
}

void generate_sine_table(uint16_t table[TABLE_SIZE]){
    for (int i = 0; i < TABLE_SIZE; i++){
        float theta = ((2.0 * M_PI * i) / TABLE_SIZE);
        float voltage = 1.5 + (1.5 * sin(theta));
        table[i] = (uint16_t)(1000 * voltage);
    }
}

void generate_sawtooth_table(uint16_t table[TABLE_SIZE]){
    for (int i = 0; i < TABLE_SIZE; i++){
        table[i] = (uint16_t)((3000 * i / TABLE_SIZE));
    }
}

void generate_triangle_table(uint16_t table[TABLE_SIZE]){
    for (int i = 0; i < TABLE_SIZE; i++){
        float edge = (float)i / TABLE_SIZE;
        float voltage;
        if(edge < 0.5){
            voltage = edge * 2.0;
        } else {
            voltage = 2.0 - (edge * 2.0);
        }
        table[i] = (uint16_t)(3000 * voltage);
    }
}

void key_logic(int pressed_key){ //Check which key is pressed and change the desired setting
while retaining the rest
    if(pressed_key == -1){ //No key pressed
        return;
    }
    else if(pressed_key == 1){
        hold_freq = 1;
    }
    else if(pressed_key == 2){
        hold_freq = 2;
    }
    else if(pressed_key == 3){
        hold_freq = 3;
    }
    else if(pressed_key == 4){
        hold_freq = 4;
    }
    else if(pressed_key == 5){
        hold_freq = 5;
    }
}

```

```

        else if(pressed_key == sine) {
            hold_waveform = sine;
        }
        else if(pressed_key == triangle) {
            hold_waveform = triangle;
        }
        else if(pressed_key == sawtooth) {
            hold_waveform = sawtooth;
        }
        else if(pressed_key == square) {
            hold_waveform = square;
        }
        else if(pressed_key == 10) { //10= *, decrease duty cycle by 10%
            if(hold_duty_cycle > 15) {
                hold_duty_cycle -= 10;
            }
        }
        else if(pressed_key == 11) { //11=#, increase duty cycle by 10%
            if(hold_duty_cycle < 85) {
                hold_duty_cycle += 10;
            }
        }
        else if(pressed_key == 0) { //Set duty cycle to 50%
            hold_duty_cycle = 50;
        }
        set_freq(hold_freq); //Set freq, if freq was not changed, this will not affect anything
        set_CCR(); //Set CCR for square waves
        switch(hold_waveform) {
            case square:
                square_wave();
                break;
            case triangle:
                triangle_wave();
                break;
            case sawtooth:
                sawtooth_wave();
                break;
            case sine:
                sine_wave();
                break;
            default:
                square_wave();
                break;
        }
        return;
    }
}

void SystemClock_Config(void);
int main(void)
{

```

```

HAL_Init();
SystemClock_Config();
RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN |
RCC_AHB2ENR_GPIOCEN);
//Generate tables for non-square waves
generate_sine_table(sine_table);
generate_sawtooth_table(sawtooth_table);
generate_triangle_table(triangle_table);
__enable_irq();
NVIC->ISER[0] = (1 << (TIM2_IRQn & 0x1F));
RCC->APB1ENR1 |= (RCC_APB1ENR1_TIM2EN); // turn on TIM2
TIM2->DIER |= (TIM_DIER_UIE | TIM_DIER_CC1IE); // enable interrupts on channel 1
TIM2->SR &= ~(TIM_SR_CC1IF | TIM_SR_UIF); //clear interrupt flag
DAC_init();
keypad_init();
set_freq(1); //set count reload value
set_CCR();
TIM2->CR1 |= TIM_CR1_CEN; //start timer
while (1)
{
    int pressed_key = check_press(0);
    key_logic(pressed_key);
}
}

void TIM2_IRQHandler(void) {
    // Handle ARR first to skip over CCR when not SQ wave
    if(TIM2->SR & TIM_SR_UIF) {
        if (wave_index >= 0){
            switch(hold_waveform) { //Pick lookup table for selected wave
                case triangle:
                    DAC_write(DAC_volt_conv(triangle_table[wave_index]));
                    break;
                case sawtooth:
                    DAC_write(DAC_volt_conv(sawtooth_table[wave_index]));
                    break;
                case sine:
                    DAC_write(DAC_volt_conv(sine_table[wave_index]));
                    break;
                default:
                    DAC_write(DAC_volt_conv(sine_table[wave_index]));
                    break;
            }
            if(++wave_index >= TABLE_SIZE) { //Check to make sure next index is not out of
range of lookup tables
                wave_index = 0;
            }
        } else { // For SQ wave
            DAC_write(DAC_volt_conv(3000)); //Set High for square waves
        }
    }
}

```

```

        TIM2->SR &= ~(TIM_SR_UIF);
    }
    // This indicates it was CCR that tripped not ARR, only for SQ wave
    else if(TIM2->SR & TIM_SR_CC1IF) {
        if(wave_index < 0) { //Check to make sure we are in a square wave
            DAC_write(DAC_volt_conv(0)); //set low for square waves
        }
        TIM2->SR &= ~(TIM_SR_CC1IF);
    }
}

```

DAC.c

```

#include "DAC.h"
#include "main.h"

void DAC_init(void) {
    //PA4 - NSS/CS, PA5 - SCK, PA7 - COPI
    RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN);
    GPIOA->AFR[0] &= ~(GPIO_AFRL_AFSEL4 | GPIO_AFRL_AFSEL5 |
        GPIO_AFRL_AFSEL7);
    GPIOA->AFR[0] |= ((5<<GPIO_AFRL_AFSEL4_Pos) |
        (5<<GPIO_AFRL_AFSEL4_Pos) |
        (5<<GPIO_AFRL_AFSEL5_Pos) |
        (5<<GPIO_AFRL_AFSEL7_Pos));
    GPIOA->MODER &= ~(GPIO_MODER_MODE4 | GPIO_MODER_MODE5 |
        GPIO_MODER_MODE7);
    GPIOA->MODER |= (GPIO_MODER_MODE4_1 | GPIO_MODER_MODE5_1 |
        GPIO_MODER_MODE7_1);

    //SPI Configure
    RCC->APB2ENR |= (RCC_APB2ENR_SPI1EN);
    SPI1->CR1 = (SPI_CR1_MSTR);
    SPI1->CR2 = (SPI_CR2_SSOE | SPI_CR2_NSSP |
        (0xF << SPI_CR2_DS_Pos));
    SPI1->CR1 |= (SPI_CR1_SPE);
}

void DAC_write(uint16_t to_write) {
    while(!(SPI1->SR & SPI_SR_TXE));
    to_write &= ~(0xF << 12);
    to_write |= (0x3 << 12);
    SPI1->DR = to_write;

    // while(!(SPI1->SR & SPI_SR_TXE));
    // while(SPI1->SR & SPI_SR_BSY);
}

uint16_t DAC_volt_conv(uint16_t mv) {
    //4096 possible values, each step = Vmax / 4096, Vmax is 3.3V for us
    if(mv > 3300) {
        return 3300*4096/3300;
    }
    return mv*4096/3300;
}

```

[1] STMicroelectronics, "UM1724: STM32 Nucleo-64 boards (MB1136) User manual," rev. 14, Aug. 2020.

[2]"MCP4901/4911/4921 Features • MCP4901: 8-Bit Voltage Output DAC • MCP4911: 10-Bit Voltage Output DAC • MCP4921: 12-Bit Voltage Output DAC • Rail-to-Rail Output • SPI Interface with 20 MHz Clock Support • Simultaneous Latching of the DAC Output with LDAC Pin • Fast Settling Time of 4.5 μ s • Selectable Unity or 2x Gain Output • External Voltage Reference Input • External Multiplier Mode • 2.7V to 5.5V Single-Supply Operation • Extended Temperature Range: -40°C to +125°C Applications • Set Point or Offset Trimming • Precision Selectable Voltage Reference • Motor Control Feedback Loop • Digitally-Controlled Multiplier/Divider • Calibration of Optical Communication Devices." Available:
<https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ProductDocuments/DataSheets/22248a.pdf>